CS 120 — INTRODUCTION TO COMPUTER SYSTEMS

Module 3

Number Systems & Data Representation

Computer Science Department · School of ICT · Copperbelt University

Dr. Z Lifelo

Academic Year 2025 / 2026

Why Number Systems Matter — The Foundation of Everything

A number System is a writing system for expressing numbers, i.e., a mathematical notation for representing numbers of a given set.

Web Colours

#003366 = R:0 G:51 B:102
Every website colour is hexadecimal

IP Addresses

192.168.1.1 = four 8-bit binary numbers
IPv4 networking uses binary octets

Cryptography

AES/RSA operate on binary blocks
256-bit key = 2^{256} possible values

File Sizes

1 KB = 1,024 bytes (why? 2^{10}=1024)
Storage uses powers of 2

Memory Addresses

CPU reads RAM at hex addresses
0x00A3FF12 — debuggers use hex daily

QR & Barcodes

QR codes encode data as binary dots
MTN MoMo payments scan binary data

Every piece of data in every computer is ultimately binary. Understanding number systems is not optional — it separates a computer SCIENTIST from a computer USER.

Module 3 — Lecture Overview

Five sections — from number system theory to encoding real-world data in binary:

A

Number Systems — Decimal, Binary, Octal & Hexadecimal

Place values · Positional notation · Why computers use binary · Hex as binary shorthand

B

Number System Conversions — Worked Examples

Decimal↔Binary · Decimal↔Hex · Hex↔Binary · Octal conversions · Verification techniques

C

Binary Arithmetic

Binary addition rules · Carry bits · Overflow · Signed vs. unsigned · Two's complement · Binary subtraction

D

Data Storage — Bits, Bytes & Units

Bit · Nibble · Byte · KB/MB/GB/TB/PB · Binary vs. SI prefixes · Storage capacity calculations

E

Character & Data Encoding

ASCII · Extended ASCII · Unicode & UTF-8 · Pixel/RGB · Image file sizes · Audio sampling · Video compression basics

MODULE 3 · SECTION A

Number Systems Decimal · Binary Octal & Hexadecimal

Place values · Positional notation · Why computers use binary · Hex shorthand

Positional Notation — The Key to ALL Number Systems

Every number system is positional: the VALUE of a digit depends on its POSITION. The base (radix) determines how many digit symbols exist and what each position is worth.

DECIMAL (Base 10) — place value breakdown of 3427_{10}:

10^3	10^2	10^1	10^0	⟷ Position Value
= 1000	= 100	= 10	= 1	
3	**4**	**2**	**7**	⟷ Face values
3000	400	20	7	

$= 3\times1000 + 4\times100 + 2\times10 + 7\times1 = 3427_{10}$

The Four Number Systems Used in Computing

System	Base	Digits Used	Subscript	Main Use
Decimal	10	0 1 2 3 4 5 6 7 8 9	10	Human-readable output
Binary	2	0, 1	2	All CPU storage and logic
Octal	8	0 1 2 3 4 5 6 7	8	Linux file permissions (chmod 755)
Hexadecimal	16	0–9 then A=10 B=11 C=12 D=13 E=14 F=15	16	Memory addresses, colour codes, debugging

Number System: Face Value, Position Value & Fractional Positions

Number System Basics

Face Value

Face value = the digit itself.
In 52_{10}: face(5)=5 and face(2)=2.

Position Value & Worked Example: 52_{10}

Position value = base^position.
In 52_{10}: 5 sits at 10^1 and 2 sits at 10^0.
So $52_{10} = 5\times10^1 + 2\times10^0 = 50 + 2 = 52$.

Key Reminder

Position changes the value contributed by a digit.
It does NOT change the digit itself.

Digits Around the Radix Point

Positions

Rightmost integer digit = position 0.
Move left: 1, 2, 3, ...
First digit right of the decimal point = −1, then −2, −3, ...

Place Values by Base

Decimal:	10^1	10^0	.	10^{-1}	10^{-2}
Octal:	8^1	8^0	.	8^{-1}	8^{-2}
Hex:	16^1	16^0	.	16^{-1}	16^{-2}

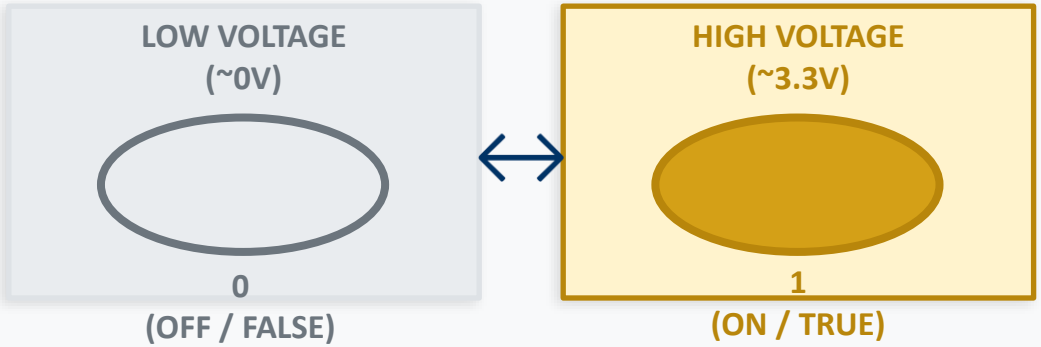
Example: 101.001_2

Positions: 2, 1, 0 . −1, −2, −3
Value = $1\times2^2 + 0\times2^1 + 1\times2^0 + 1\times2^{-3}$
= 4 + 1 + 1/8 = 5.125_{10}

Why Computers Use Binary — Two States, Infinite Possibilities

There is no mysticism — binary is the most reliable and efficient way to represent information in electronic hardware.

FIGURE: A transistor has exactly 2 stable states:



Why Binary is Ideal for Hardware

- ✓ Transistors naturally have 2 states (on/off) — easy to build
- ✓ Noise tolerant: small voltage fluctuations don't flip 0→1
- ✓ Boolean algebra (AND, OR, NOT) maps perfectly to binary
- ✓ Billions of binary transistors fit on a 3nm chip
- ✓ Error detection: easy to find and correct single-bit errors

An 8-bit binary number — each cell is one transistor (ON=1, OFF=0):

0	1	0	0	0	0	1	0
2^7=128	2^6=64	2^5=32	2^4=16	2^3=8	2^2=4	2^1=2	2^0=1

$= 0\times128 + 1\times64 + 0\times32 + 0\times16 + 0\times8 + 0\times4 + 1\times2 + 0\times1 = 64 + 2 = 66_{10} =$ ASCII 'B'

Gold cells = 1-bits that contribute their place value. White cells = 0-bits that contribute nothing.

KEY INSIGHT: $01000010_2 = 66_{10} = 42_{16}$ = ASCII 'B'. Same binary pattern — three different representations. Context determines the meaning.

Hexadecimal — Binary's Compact Human Notation

Binary is the computer's language. Hex is the human's shortcut. ONE hex digit = EXACTLY 4 binary bits (a nibble). One byte = exactly 2 hex digits. No arithmetic to convert — just substitute.

The Complete Hexadecimal Reference Table:

Hex	Binary	Dec	Hex	Binary	Dec
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	A	1010	10
3	0011	3	B	1011	11
4	0100	4	C	1100	12
5	0101	5	D	1101	13
6	0110	6	E	1110	14
7	0111	7	F	1111	15

KEY RULE: To convert hex↔binary: substitute each hex digit with exactly 4 binary bits. One hex digit = 4 bits = 1 nibble. Always group from the RIGHT.

Why Hex Exists — Worked Examples

Memory Address: 0x00A3FF12

$= 0000\ 0000\ 1010\ 0011\ 1111\ 1111\ 0001\ 0010_2$

32 binary digits → only 8 hex digits ✓ Compact!

Colour Code: #003366

00_{16}=R:0 33_{16}=G:51 66_{16}=B:102

3 bytes, 6 hex digits vs. 24 binary digits ✓

Converting: Hex → 4-bit Binary (direct substitution)

$B4F_{16}$ → B=1011, 4=0100, F=1111

$= 1011\ 0100\ 1111_2$ (12 bits, no arithmetic!)

Converting: Binary → Hex (group in 4s from right)

10110101_2 → 1011=B, 0101=5 → $B5_{16}$ ✓

MODULE 3 · SECTION B

Number System Conversions — Worked Examples

Decimal↔Binary · Decimal↔Hex · Binary↔Hex · Octal · Integer fraction conversions. Verification

Decimal → Binary: Repeated Division Method

Divide by 2 repeatedly. Record remainders. Read them BOTTOM TO TOP. That is your binary result.

Example 1: Convert 42_{10} to Binary

42 ÷ 2	= 21	R = 0	← LSB (rightmost)
21 ÷ 2	= 10	R = 1	
10 ÷ 2	= 5	R = 0	
5 ÷ 2	= 2	R = 1	
2 ÷ 2	= 1	R = 0	
1 ÷ 2	= 0	R = 1	← MSB (leftmost)

Remainders ↑ (bottom→top): 101010_2 = 42_{10} ✓

Verify: 1×32 + 0×16 + 1×8 + 0×4 + 1×2 + 0×1 = 32+8+2 = 42 ✓

SHORTCUT — Powers of 2 in binary:
8=1000 · 16=10000 · 32=100000 · 64=1000000 · 128=10000000 · 256=100000000

Example 2: Convert 200_{10} to Binary

200 ÷ 2	= 100	R= 0	← LSB
100 ÷ 2	= 50	R= 0	
50 ÷ 2	= 25	R= 0	
25 ÷ 2	= 12	R= 1	
12 ÷ 2	= 6	R= 0	
6 ÷ 2	= 3	R= 0	
3 ÷ 2	= 1	R= 1	
1 ÷ 2	= 0	R= 1	← MSB

Bottom → Top: 11001000_2 = 200_{10} ✓

Verify: 128+64+8 = 200 ✓

Binary → Decimal: The Place Value Method

Multiply each bit by its place value (power of 2) and sum the results. Only 1-bits contribute to the total.

MEMORISE these powers of 2:

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
128	64	32	16	8	4	2	1

Example 1: Convert 10110010_2 to Decimal

1	0	1	1	0	0	1	0
+128	0	+32	+16	0	0	+2	0

$= 128 + 0 + 32 + 16 + 0 + 0 + 2 + 0 = 178_{10}$

Example 2: Convert 11111111_2 — the most important binary number!

1	1	1	1	1	1	1	1

$= 128+64+32+16+8+4+2+1 = 255_{10}$ ← MAXIMUM value in a single byte (8 bits)

255 = FF_{16} = 11111111_2 = max RGB channel value (#FFFFFF = white · #000000 = black)

Decimal ↔ Hexadecimal Conversions

Decimal → Hex: Divide by 16

Example 1: Convert 255_{10} to Hex

255 ÷ 16 = 15 R= 15 = F ← LSD

15 ÷ 16 = 0 R= 15 = F ← MSD

Remainders ↑: FF_{16} = 255_{10} ✓

Example 2: Convert 3427_{10} to Hex

3427 ÷ 16 = 214 R= 3 ← LSD

214 ÷ 16 = 13 R= 6

13 ÷ 16 = 0 R= 13 = D ← MSD

Bottom → Top: $D63_{16}$ = 3427_{10} ✓

Hex → Decimal: Place Value Method

Example 3: Convert $2A_{16}$ to Decimal

Position: 16^1 16^0

Digit: 2 A (A=10)

Value: $2\times16 + 10\times1$

= 32 + 10 = 42_{10} ✓

Example 4: Convert $1F4_{16}$ to Decimal

Position: 16^2 16^1 16^0

Digit: 1 F(15) 4

Value: $1\times256 + 15\times16 + 4\times1$

= 256 + 240 + 4 = 500_{10} ✓

Powers of 16 — Memorise These

$16^0 = 1$ · $16^1 = 16$ · $16^2 = 256$

$16^3 = 4{,}096$ · $16^4 = 65{,}536$

CROSS-CHECK: $2A_{16} = 42_{10} = 101010_2$ (same number, 3 bases!)

Hex ↔ Binary & Octal Conversions

Hex ↔ Binary: Direct 4-bit Substitution

Each hex digit = exactly 4 binary bits. Just substitute — NO arithmetic.

Example: Convert $3AF0_{16}$ to Binary:

3	A	F	0
0011	1010	1111	0000

$3AF0_{16} = 0011\ 1010\ 1111\ 0000_2$ ✓

Example: 1011010110001_2 → Hex (group in 4s from RIGHT):

1011010110001_2 → pad left: 0001 0110 1011 0001_2

= 0001=1, 0110=6, 1011=B, 0001=1

= $16B1_{16}$ ✓ (group from RIGHT, pad LEFT with 0s)

Octal (Base 8): 3-bit Groups

Octal = 3 binary bits per digit (2^3=8 symbols: 0–7). Used in Linux file permissions.

Binary → Octal: group in 3s from RIGHT

$110\ 111\ 010_2$ (group in 3s from right)

110=6, 111=7, 010=2 → 672_8 ✓

Octal → Binary: expand each digit to 3 bits

524_8 → 5=101, 2=010, 4=100

= 101 010 100_2 = 101010100_2 ✓

Real World: Linux File Permissions (chmod)

drwxr-xr-x ← directory listing

Owner: rwx = 111_2 = 7_8

Group: r-x = 101_2 = 5_8

Other: r-x = 101_2 = 5_8

chmod 755 filename ← octal permission!

Decimal Fractions → Binary, Octal & Hexadecimal

Method: Multiply by the Target Base

Core Rule

Multiply the fractional part by the target base.
Binary ×2 · Octal ×8 · Hex ×16

Step Pattern

1. Multiply the fractional part by the target base.
2. Split the result into whole part + new fraction.
3. Record the whole part.
4. Repeat with the new fraction.
5. Read recorded digits TOP → BOTTOM.

Stopping Rule

Stop when the fraction becomes 0, or when you have enough places for the required precision.

Worked Example: 0.2345_{10} → Binary

Repeated Multiplication

$$
\begin{aligned}
0.2345 \times 2 &= 0.4690 \rightarrow 0 \\
0.4690 \times 2 &= 0.9380 \rightarrow 0 \\
0.9380 \times 2 &= 1.8760 \rightarrow 1 \\
0.8760 \times 2 &= 1.7520 \rightarrow 1 \\
0.7520 \times 2 &= 1.5040 \rightarrow 1 \\
0.5040 \times 2 &= 1.0080 \rightarrow 1
\end{aligned}
$$

Approximation

$(0.2345)_{10} \approx (0.001111)_2$
If the expansion does not terminate, state the result as approximate.

Same method works for octal (×8) and hexadecimal (×16).

Decimal Numbers with Integer and Fraction Parts

Convert the Two Parts Separately

Step 1: Split the Number

Example: 34.4674_{10}
Integer part = 34
Fraction part = 0.4674
These two parts use different algorithms.

Step 2: Convert the Integer Part

Use repeated division exactly as before.

$34_{10} = 100010_2$

$34_{10} = 42_8 = 22_{16}$

Step 3: Convert the Fraction Part

Use repeated multiplication by 2, 8, or 16.
Keep only the new fractional part after each step.

How the Final Answer is Formed

Step 4: Join the Two Results

Binary: $100010.xxxxx_2$
Octal: $42.xxxx_8$
Hex: $22.xxxx_{16}$

Key Idea

Whole-number conversion and fractional conversion are not the same process.
Convert each part separately, then combine them with the radix point.

Practice Prompt

Try 34.4674_{10} → binary and 34.4674_{10} → octal using the split-then-combine method.

Fractions in Binary, Octal & Hex → Decimal

Use Negative Powers of the Base

General Rule

Digits left of the point use positions 0, 1, 2, ...
Digits right of the point use −1, −2, −3, ...
Sum digit × base^position for ALL digits.

Binary Example: 1011.1001_2

$$1011.1001_2$$
$$= 1\times2^3 + 0\times2^2 + 1\times2^1 + 1\times2^0 + 1\times2^{-1} + 0\times2^{-2} + 0\times2^{-3} + 1\times2^{-4}$$
$$= 8 + 0 + 2 + 1 + 1/2 + 1/16$$
$$= 11.5625_{10}$$

Hexadecimal Examples

Example 1: $.15_{16}$

$$.15_{16} = 1\times16^{-1} + 5\times16^{-2}$$
$$= 1/16 + 5/256 = 21/256$$
$$\approx 0.08203125_{10}$$

Example 2: $4D.21_{16}$

$$4D.21_{16} = 4\times16^1 + 13\times16^0 + 2\times16^{-1} + 1\times16^{-2}$$
$$= 64 + 13 + 1/8 + 1/256$$
$$= 77.12890625_{10}$$

Reminder

Octal fractions use the same negative-position rule.
Shortcut reminder: octal = 3-bit groups, hex = 4-bit groups.

Conversion Summary — Quick Reference & Must-Know Values

MEMORISE this table.

Decimal	Binary	Hexadecimal	Octal
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7
8	1000	8	10
9	1001	9	11
10	1010	A	12
11	1011	B	13
12	1100	C	14
13	1101	D	15
14	1110	E	16
15	1111	F	17
16	10000	10	20
32	100000	20	40
64	1000000	40	100
128	10000000	80	200
255	11111111	FF	377

Must-Know Values

$15 = F = 1111_2$
Max nibble

$127 = 7F_{16}$
Max signed byte (+)

$128 = 80_{16}$
Min signed byte (−)

$255 = FF = 11111111_2$
Max unsigned byte

$256 = 100_{16} = 2^8$
256 total byte values

65535 = FFFF
Max 16-bit unsigned

MODULE 3 · SECTION D

Data Storage — Bits, Bytes & Units

Bit · Nibble · Byte · KB/MB/GB/TB/PB · Binary vs. SI prefixes

Bits, Bytes & the Storage Hierarchy

Unit	Definition	Example
BIT	Single binary digit: 0 or 1. The smallest unit of digital data.	*Transistor state: ON or OFF*
NIBBLE	4 bits = 1 hex digit. Half a byte.	*$F_{16} = 1111_2$ = one nibble*
BYTE	8 bits — the basic addressable unit of storage in almost all computers.	*1 ASCII character = 1 byte*
KILOBYTE	1,024 bytes = 2^{10} bytes. (SI: 1,000 bytes = 1 kB)	*Small text document ≈ 2–5 KB*
MEGABYTE	1,048,576 bytes = 2^{20} bytes.	*MP3 song ≈ 3–5 MB*
GIGABYTE	1,073,741,824 bytes = 2^{30} bytes.	*HD film ≈ 2–8 GB*
TERABYTE	2^{40} bytes.	*Laptop drive 1–4 TB typical*
PETABYTE	2^{50} bytes.	*Facebook stores ~105 PB photos*

Warning: Manufacturers use SI (×1,000): 1 KB=1,000 bytes. OS uses binary (×1,024): 1 KiB=1,024 bytes. A '1 TB' drive shows as 931 GB in Windows — not a scam, just different definitions!

Tutorial Discussion — Module 3 Calculations

Individual work or pairs — show ALL working — 15 minutes

1

(a) 157_{10} → binary (b) 10110111_2 → decimal (c) AC_{16} → decimal (d) 243_{10} → hex (e) $3F2_{16}$ → binary (f) 101110_2 → hex

Conversions

💡 *(a) repeated ÷2. (b) place values. (c) A=10,C=12. (d) repeated ÷16. (e) each hex digit → 4 bits. (f) group in 4s from right.*

Module 3 — Conversion Methods & Key Formulas

All six conversion methods. Know these without referring to notes.

Decimal → Binary

1. Divide by 2 repeatedly
2. Record remainders
3. Read BOTTOM → TOP

→ e.g. 42 → 101010_2

Binary → Decimal

1. Write place values (128,64...1)
2. Multiply each bit × its value
3. Sum the 1-bit contributions

→ e.g. 10110 → $16+4+2=22_{10}$

Decimal → Hex

1. Divide by 16 repeatedly
2. Remainders ≥10: use A-F
3. Read BOTTOM → TOP

→ e.g. 255 → FF_{16}

Hex → Decimal

1. Write hex place values (256,16,1)
2. Multiply digit × place (A=10...F=15)
3. Sum the products

→ e.g. $2A_{16} = 32+10 = 42_{10}$

Hex ↔ Binary

1. One hex digit = exactly 4 bits
2. Substitute directly (no arithmetic!)
3. Binary→Hex: group in 4s from RIGHT

→ e.g. $AF_{16} = 10101111_2$

Module 3 in Professional Practice — Zambian Applications

Number systems appear in every area of professional computing:

Web Development

- Colour codes: background-color: #003366 — you are writing hex data
- CSS rgba(0,163,186,0.8) — decimal RGB + opacity (alpha channel value)
- Optimise image sizes for Zambian mobile users — WebP, lazy loading, compression
- Base64 encoding: embed small images in CSS/HTML as encoded binary strings

Network Engineering

- IP: 192.168.1.1 = 11000000.10101000.00000001.00000001 (4 binary octets)
- Subnet mask: /24 = 255.255.255.0 = all-1s then all-0s in binary
- MAC address: 00:1A:2B:3C:4D:5E — six hex bytes, hardware identifier
- MTN Zambia/Airtel network engineers configure these daily

Database & Backend

- Always use UTF-8 (utf8mb4 in MySQL) for Zambian student name fields
- BLOB data type: stores images/audio as binary in database tables
- Bit flags: store 8 boolean values in 1 byte (e.g. user permissions)
- ZRA taxpayer database: storage planning requires capacity calculations

Security & Crypto

- AES encryption operates on 128/256-bit binary blocks
- SHA-256: converts any input to a 256-bit hex string (64 hex chars)
- MTN MoMo PIN stored as binary hash — never plain text
- XOR in binary = foundation of stream ciphers and OTP encryption

Key Takeaways — Module 3

1

Binary is the language of hardware

Transistors have exactly 2 stable states. Every number, character, colour, image, and sound stored in any computer is ultimately binary. Not mystical — just physics.

2

All systems share positional notation

Binary (base 2), octal (base 8), decimal (base 10), hex (base 16) all work the same way — only the base changes. Master the principle once; apply it everywhere.

3

Hex is binary in compact human form

1 hex digit = exactly 4 bits. Colour codes, memory addresses, MAC addresses, debugging output — all hex. It exists purely to make binary readable for humans.

4

A byte holds exactly 256 values (0–255)

$11111111_2 = 255_{10} = FF_{16}$. The single most important binary value. RGB channels, ASCII, subnet masks, 8-bit registers — all use this range.

Can You Now...? — Module 3 Learning Outcomes Self-Check

Tick only what you can genuinely do without notes. Unticked items = study priorities before Quiz 3.

✓	Explain WHY computers use binary (two voltage states, hardware simplicity)
	Convert any decimal number to binary using repeated division
✓	Convert any binary number to decimal using place values
✓	Convert decimal to hexadecimal and back using repeated division
✓	Convert between hex and binary using 4-bit group substitution
✓	Convert between binary and octal using 3-bit groups
✓	Convert decimal fractions to binary, octal, hex and vice versa

Common Mistakes & Exam Tips — Module 3

✗ Reading division remainders TOP → BOTTOM

✓ Always read BOTTOM → TOP. Draw a big upward arrow beside your remainder column.

✗ Using 1000 instead of 1024 for KB in CS

✓ In computing: 1 KB=1,024 bytes. Use 1024 for CS calculations unless stated otherwise.

✗ Confusing bit (b) and byte (B)

✓ Speeds use bits/sec (Mbps). Storage uses Bytes (MB). A 10 Mbps link transfers 1.25 MB/sec.

✗ No base subscript on numbers

✓ $42_{10} \neq 42_{2} \neq 42_{16}$. ALWAYS write the subscript in exam answers. Ambiguous = no marks.

✗ Grouping binary for hex from LEFT not RIGHT

✓ ALWAYS start from the RIGHTMOST bit. Pad with leading zeros on the LEFT if needed.

✗ No verification of conversion answer

✓ ALWAYS check by converting back. 10 seconds saves marks. Method error ≠ concept error.

GOLDEN RULE: Show ALL working. A wrong final answer with correct method earns partial marks. A right answer with no working earns zero.